

Serveur MCP d'outils d'analyse symbolique

J3 - IA symbolique : SAT/SMT, preuves par résolution, raisonnement OWL

Quentin Lauret

EPITA

16 juillet 2026

Plan

- 1 Contexte et objectifs
- 2 Architecture
- 3 Les trois outils symboliques
- 4 Interface MCP et orchestration LLM
- 5 Évaluation
- 6 Conclusion

Contexte : pourquoi un serveur MCP symbolique ?

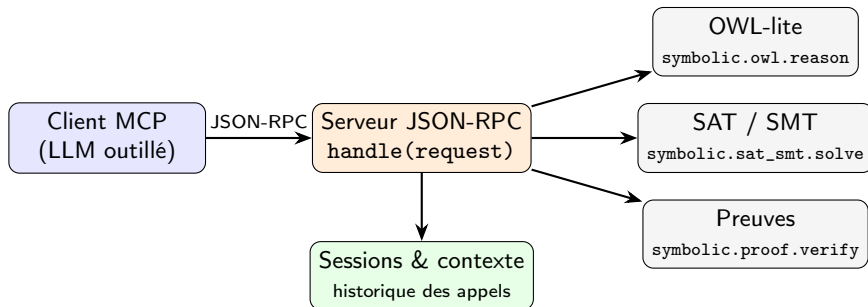
- Les LLM raisonnent mal sur la logique formelle : hallucinations, erreurs de calcul, preuves invalides.
- Idée : **déléguer le raisonnement formel** à des outils symboliques exacts, exposés derrière une interface standard.
- Le **Model Context Protocol (MCP)** définit ce contrat : un client (LLM) découvre et appelle des outils via JSON-RPC.

Objectif du projet

Implémenter un serveur JSON-RPC inspiré de MCP qui encapsule trois raisonneurs symboliques, avec gestion de sessions, exemples d'usage et évaluation de précision / robustesse.

Choix technique : Python pur, zéro dépendance obligatoire - livrable pédagogique, reproductible et exécutable partout.

Architecture générale



- 1 Le client envoie une requête JSON-RPC (`initialize`, `tools/list`, `tools/call`).
- 2 Le serveur valide la méthode et la session, puis route vers l'outil.
- 3 L'outil renvoie `structuredContent` (`machine`) + `content` (texte pour LLM).
- 4 L'appel et son résultat sont mémorisés dans le contexte de session.

Outil 1 - Solveur SAT/SMT

Deux familles d'entrées :

- **SAT** : formule CNF en clauses d'entiers, résolue par **DPLL** (propagation unitaire + littéraux purs).
- **SMT borné** : contraintes arithmétiques sur entiers avec bornes finies, résolues par énumération.

```
# SAT : (P ou Q) et (~P ou Q) et (P ou ~Q)
{"clauses": [[1, 2], [-1, 2], [1, -2]], "symbols": ["P", "Q"]}
# -> sat, modele {P: true, Q: true}

# SMT borne
{"constraints": ["x + y == 7", "x >= 2", "y >= 2", "x < y"],
 "bounds": {"x": [0, 10], "y": [0, 10]}}
# -> sat, modele {x: 2, y: 5}
```

Sécurité : les expressions SMT sont parsées avec le module ast - aucun appel de fonction ni accès système autorisé.

Outil 2 - Vérificateur de preuves

Vérifie des preuves par **résolution propositionnelle** : chaque étape doit dériver une clause par résolution de deux clauses précédentes. La clause vide [] prouve l'insatisfiabilité.

```
# Premises : (P ou Q), ~P, ~Q -- refutation en 2 etapes
{"premises": [{"P", "Q"}, [{"~P"}, [{"~Q"}]],
 "proof": [
   {"id": "S1", "rule": "resolve", "from": [{"P1", "P2"}], "clause": [{"Q"}]},
   {"id": "S2", "rule": "resolve", "from": [{"S1", "P3"}], "clause": []}
 ],
 "conclusion": []}
# -> valid: true
```

- Notations acceptées : "P", "~P", "not P", $\neg P$, clauses en liste ou en chaîne.
- Chaque étape est **contrôlée formellement** : une résolution incorrecte invalide la preuve (pas de confiance aveugle).

Saturation de triplets (sujet, prédicat, objet) sur un fragment OWL/RDFS :

- transitivité de `subClassOf`, `subPropertyOf`
- propagation des types et assertions
- `equivalentClass`, `inverseOf`
- `domain`, `range`, `transitiveProperty`
- incohérences via `disjointWith`

```
["Dog", "subClassOf", "Mammal"]  
["Mammal", "subClassOf", "Animal"]  
["fido", "type", "Dog"]  
["parentOf", "inverseOf", "childOf"]  
["alice", "parentOf", "bob"]  
# Inferences :  
# fido type Animal  
# bob childOf alice
```

Point à saturation avec garde-fou (`max_iterations`); le résultat distingue triplets de base et triplets **inférés**.

Interface JSON-RPC / MCP

Méthodes exposées :

- initialize
- tools/list : schémas JSON des 3 outils
- tools/call : routage + validation
- sessions/create, sessions/get

```
{ "jsonrpc": "2.0", "id": 4,  
  "method": "tools/call",  
  "params": {  
    "session_id": "...",  
    "name": "symbolic.sat_smt.solve",  
    "arguments": {  
      "clauses": [[1,2],[-1,2],[1,-2]],  
      "symbols": ["P","Q"]} }}
```

Double sortie par outil :

- structuredContent : exploitable par programme
- content : résumé textuel pour le LLM

Chaque outil est documenté par un `inputSchema` JSON - le client découvre dynamiquement les capacités du serveur.

Traduction bidirectionnelle LLM $\leftrightarrow$ outils

- `llm_to_tool_call(message)` : un message en langage naturel est routé vers l'outil pertinent (détection d'intention simple par mots-clés : *preuve, ontologie, contrainte...*).
- `tool_result_to_llm_answer(response)` : le résultat symbolique structuré est converti en réponse textuelle exploitable dans un dialogue.
- Le **contexte de session** conserve chaque appel : un orchestrateur peut réutiliser les résultats précédents, tracer les décisions et construire un raisonnement multi-étapes.

Exemple

« Peux-tu vérifier cette ontologie OWL ? »

⇒ appel `symbolic.owl.reason` ⇒ « L'ontologie est cohérente, 6 triplets inférés... »

Évaluation : précision et robustesse

Précision - 8 benchmarks de référence, **8/8 réussis (100 %)** :

Benchmark	Outil	Attendu
sat_cnf_sat / sat_cnf_unsat	SAT	sat / unsat
smt_sat / smt_unsat	SMT borné	modèle valide / unsat
proof_valid_refutation	Preuves	preuve acceptée
proof_invalid_step	Preuves	étape incorrecte rejetée
owl_type_propagation	OWL	fido type Animal inféré
owl_disjoint_inconsistency	OWL	incohérence détectée

Robustesse - entrées malveillantes ou invalides $\Rightarrow$ erreurs JSON-RPC propres (pas de crash) : outil inconnu, tentative d'injection `__import__('os').system(...)`, version JSON-RPC invalide.

Limites assumées

- SMT borné sur entiers uniquement (énumération)
- Fragment OWL-lite, pas OWL DL complet
- Boucle de transport stdio non lancée dans le notebook

Vers la production

- Remplacer le mini-SMT par **Z3**
- Raisonneur OWL via **Owlready2** / RDFLib / OWL-RL
- Transport MCP complet (stdio ou HTTP)

Bilan

Un serveur MCP fonctionnel qui montre comment donner à un LLM un accès fiable à du raisonnement symbolique exact : découverte d'outils, appels typés, sessions, et évaluation à 100 % de précision sur les benchmarks - le tout en Python pur, sans dépendance.

Merci !

Questions ?